

Отчёт по лабораторной работе №6

Автоматизированное функциональное тестирование веб-сайта с помощью фреймворка Selenium

1. Описание предмета тестирования

Название сайта: Додо Пицца (<https://dodopizza.ru>)

Назначение: Сервис заказа пиццы и других блюд (закуски, напитки, десерты) с доставкой или самовывозом. Пользователь может выбрать город, добавить товары в корзину, оформить заказ, отслеживать его статус.

Основные функции для пользователя:

Выбор города (автоматическое определение или ручной ввод);

Просмотр меню с фильтрацией по категориям;

Добавление товаров в корзину;

Настройка состава пиццы (тесто, размер, дополнительные ингредиенты);

Оформление заказа с выбором адреса, времени, способа оплаты;

Просмотр статуса текущего заказа (для авторизованных).

2. Описание окружения тестирования

Параметр	Значение
Оборудование	Процессор: AMD Ryzen 5 4500U with Radeon Graphics, ОЗУ: 8 ГБ, экран: 1920×1080
ОС	Windows 11 Pro, версия 25H2, сборка ОС 26200.7627
Браузер	Google Chrome версия 146.0.7680.165 (официальная сборка) (64 бит)
Расширения	Отсутствуют
Антивирус	Windows Defender (активен)

Сеть	Wi-Fi 5, 100 Мбит/с (стабильный сигнал)
------	---

3. Сценарии использования

Напишем общий код для выполнения всех трёх тестовых сценариев из прошлой работы.

```
import os
import sys
import time
from pathlib import Path

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import (
    TimeoutException,
    ElementClickInterceptedException,
    StaleElementReferenceException,
    WebDriverException,
)

sys.stdout.reconfigure(encoding="utf-8")

BASE_URL = "https://dodopizza.ru"
WAIT = 10
DEFAULT_CITY = "Москва"
TARGET_CITY = "Пермь"

def setup_driver():
    print("[ INFO ] Connecting to Chrome on port 9222...")
    options = webdriver.ChromeOptions()
    options.debugger_address = "127.0.0.1:9222"

    # Не ждём полную загрузку страницы
    options.page_load_strategy = "none"

    try:
        driver = webdriver.Chrome(options=options)
        driver.set_page_load_timeout(15)
        driver.set_script_timeout(15)
        driver.set_window_size(1920, 1080)

        driver.get(BASE_URL)
        time.sleep(2)
        print(f"[ SUCCESS ] Connected! URL: {driver.current_url}")
        return driver
    except Exception as e:
        print(f"[ ERROR ] {e}")
```

```

        return None

def make_screenshot(driver, name):
    d = os.path.join(os.path.dirname(os.path.abspath(__file__)),
"screenshots")
    os.makedirs(d, exist_ok=True)
    p = os.path.join(d, name)
    driver.save_screenshot(p)
    print(f"[ INFO ] Screenshot: {p}")

def jc(driver, el):
    driver.execute_script("arguments[0].click();", el)

def wait_for(driver, condition, timeout=WAIT):
    return WebDriverWait(driver, timeout).until(condition)

def wait_overlay_disappear(driver, timeout=3):
    try:
        WebDriverWait(driver, timeout).until(
            EC.invisibility_of_element_located((By.CSS_SELECTOR, ".popup-fade")))
    except Exception:
        pass

def safe_click(driver, element):
    try:
        element.click()
    except (ElementClickInterceptedException, WebDriverException,
StaleElementReferenceException):
        driver.execute_script("arguments[0].click();", element)

def click_first(driver, locators, timeout=WAIT):
    last_error = None
    for by, value in locators:
        try:
            el = wait_for(driver, EC.presence_of_element_located((by,
value))), timeout)
            wait_overlay_disappear(driver, timeout=2)

            driver.execute_script("arguments[0].scrollIntoView({block:'center'});", el)
            time.sleep(0.2)
            try:
                wait_for(driver, EC.element_to_be_clickable((by, value)),
timeout=2)
            except Exception:
                pass
            safe_click(driver, el)
            return el
        except Exception as e:

```

```

        last_error = e
    raise last_error

def type_first(driver, locators, text, timeout=WAIT, clear=True):
    last_error = None
    for by, value in locators:
        try:
            el = wait_for(driver, EC.visibility_of_element_located((by,
value)), timeout)
            wait_overlay_disappear(driver, timeout=2)

driver.execute_script("arguments[0].scrollIntoView({block:'center'})";, el)
            time.sleep(0.2)
            if clear:
                try:
                    el.clear()
                except Exception:
                    pass
            el.send_keys(text)
            return el
        except Exception as e:
            last_error = e
    raise last_error

def text_exists(driver, pattern):
    pattern = pattern.lower()
    xpath = (
        "//*[contains("
        "translate(normalize-space(.), "
        "'АБВГДЕЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯАВСДЕFGHIJ KLMNOPQRSTUVWXYZ', "
        "'абвгдежзийклмнопрстуфхцчшщъыьэюяаbcdefghijklmnopqrstuvwxyz'), "
        f"'{pattern}'))]"
    )
    return len(driver.find_elements(By.XPATH, xpath)) > 0

def open_home(driver):
    driver.get(BASE_URL)
    wait_for(driver, EC.presence_of_element_located((By.TAG_NAME, "body")),
timeout=5)
    time.sleep(1)
    auto_select_city(driver, DEFAULT_CITY)

def auto_select_city(driver, city_name="Москва"):
    wait_overlay_disappear(driver, timeout=2)

    # Сначала пробуем закрыть/подтвердить попап
    confirm_buttons = [
        (By.XPATH, "//*[self::button or self::a][contains(., 'Да,
верно')]"),
        (By.XPATH, "//*[self::button or self::a][contains(., 'Верно')]"),
        (By.XPATH, "//*[self::button or self::a][contains(., 'Ок')]"),
        (By.XPATH, "//*[self::button or self::a][contains(., 'Принять')]"),
    ]

```

```

        (By.XPATH, "//*[self::button or self::a][contains(.,
'Продолжить')]]"),
    ]
    try:
        click_first(driver, confirm_buttons, timeout=2)
        time.sleep(1)
        wait_overlay_disappear(driver, timeout=2)
        return
    except Exception:
        pass

    # Открываем выбор города
    open_city_buttons = [
        (By.XPATH, "//*[self::button or self::a][contains(., 'Выберите
город')]]"),
        (By.XPATH, "//*[self::button or self::a][contains(., 'Сменить
город')]]"),
        (By.XPATH, "//*[self::button or self::a][contains(., 'Москва')]]"),
        (By.XPATH, "//*[self::button or self::a][contains(., 'Город')]]"),
    ]
    try:
        click_first(driver, open_city_buttons, timeout=3)
    except Exception:
        pass

    time.sleep(0.5)
    wait_overlay_disappear(driver, timeout=2)

    city_input_locators = [
        (By.XPATH, "///input[contains(@placeholder, 'город')]]"),
        (By.XPATH, "///input[contains(@placeholder, 'Поиск')]]"),
        (By.XPATH, "///input[@type='text']"),
        (By.XPATH, "///input"),
    ]

    inp = None
    for by, value in city_input_locators:
        items = driver.find_elements(by, value)
        if items:
            inp = items[0]
            break

    if inp is not None:
        try:
            driver.execute_script("arguments[0].scrollIntoView({block:'center'});",
inp)

            time.sleep(0.2)
            inp.clear()
            inp.send_keys(city_name)
            time.sleep(1)
            inp.send_keys(Keys.ENTER)
            time.sleep(1)
            wait_overlay_disappear(driver, timeout=2)
            return
        except Exception:

```

```

        pass

city_click_locators = [
    (By.XPATH, f"//*[contains(., '{city_name}')]"),
    (By.XPATH, "//*[contains(., 'Москва')]"),
]
try:
    click_first(driver, city_click_locators, timeout=3)
    time.sleep(1)
except Exception:
    pass

try:
    overlays = driver.find_elements(By.CSS_SELECTOR, ".popup-fade")
    for overlay in overlays:
        driver.execute_script("arguments[0].remove();", overlay)
except Exception:
    pass

def add_to_cart(driver):
    add_locators = [
        (By.XPATH, "//button[contains(., 'В корзину')]"),
        (By.XPATH, "//button[contains(., 'Добавить')]"),
        (By.XPATH, "//button[contains(., 'P')]"),
    ]
    click_first(driver, add_locators, timeout=10)

def open_cart(driver):
    wait_overlay_disappear(driver, timeout=2)

    cart_locators = [
        (By.XPATH, "//button[contains(@aria-label, 'Корзина')]"),
        (By.XPATH, "//button[.//bdi[@data-part='price-label']]"),
        (By.XPATH, "//a[contains(., 'Корзина')]"),
    ]

    click_first(driver, cart_locators, timeout=8)

def run_scenario_1(driver):
    print("\n=== Scenario 1: Add pizza to cart ===")
    pepperoni_locators = [
        (By.XPATH, "//span[normalize-space()='Пепперони']")
    ]
    try:
        click_first(driver, pepperoni_locators, timeout=5)
    except Exception:
        print("[ WARNING ] Pepperoni not found, continue")
    time.sleep(2)
    make_screenshot(driver, "1. step1_pizza_opened.png")

    time.sleep(2)

    size_locators = [

```

```

        (By.XPATH, "//label[contains(@data-testid, 'pizza_size_35')]")
    ]
    try:
        click_first(driver, size_locators, timeout=5)
        print("[ INFO ] Size 35 cm selected")
        make_screenshot(driver, "1. step2_size_35.png")
        time.sleep(5)
    except Exception:
        print("[ WARNING ] Size 35 cm not found, continue")

    add_to_cart(driver)
    print("[ INFO ] Added to cart")
    make_screenshot(driver, "1. step3_added_to_cart.png")
    time.sleep(1)

    open_cart(driver)
    time.sleep(1)
    make_screenshot(driver, "1. step4_cart_opened.png")

    if text_exists(driver, "пепперони"):
        print("[ SUCCESS ] Scenario 1 done")
    else:
        raise AssertionError("Пицца 'Пепперони' не найдена в корзине")

def run_scenario_2(driver):
    print("\n=== Scenario 2: Change city ===")
    time.sleep(3)
    city_button_locators = [
        (By.XPATH, "//button[@data-testid='header__about-slogan-
text_link']")
    ]
    click_first(driver, city_button_locators, timeout=5)
    time.sleep(1)
    make_screenshot(driver, "2. step1_city_popup_opened.png")

    city_input_locators = [
        (By.XPATH, "//input[contains(@placeholder, 'город')]"),
        (By.XPATH, "//input[contains(@placeholder, 'Поиск')]"),
        (By.XPATH, "//input"),
    ]
    type_first(driver, city_input_locators, TARGET_CITY, timeout=5)
    time.sleep(1)
    make_screenshot(driver, "2. step2_city_typed.png")
    time.sleep(1)

    choose_city_locators = [
        (By.XPATH, f"//a[normalize-space()='{{TARGET_CITY}}']"),
    ]
    click_first(driver, choose_city_locators, timeout=5)
    time.sleep(7)
    make_screenshot(driver, "2. step3_city_selected.png")

    if text_exists(driver, "перм"):
        print("[ SUCCESS ] Scenario 2 done")
    else:

```

```

        raise AssertionError("Город 'Пермь' не отображается на странице")

def run_scenario_3(driver):
    print("\n=== Scenario 3: Invalid promo code ===")
    time.sleep(1)
    pepperoni_locators = [
        (By.XPATH, "//span[normalize-space()='Пепперони']")
    ]
    try:
        click_first(driver, pepperoni_locators, timeout=5)
    except Exception:
        print("[ WARNING ] Pepperoni not found, continue")

    add_to_cart(driver)
    open_cart(driver)
    time.sleep(1)
    make_screenshot(driver, "3. step1_cart_for_promo.png")

    promo_input_locators = [
        (By.XPATH, "//input[contains(@placeholder, 'Промокод')]"),
        (By.XPATH, "//input[contains(@placeholder, 'промокод')]"),
        (By.XPATH, "//input"),
    ]
    type_first(driver, promo_input_locators, "НЕВЕРНЫЙКОД123", timeout=5)
    print("[ INFO ] Promo code entered")
    make_screenshot(driver, "3. step2_promo_typed.png")

    apply_locators = [
        (By.XPATH, "//button[contains(., 'Применить')]"),
        (By.XPATH, "//*[self::button or self::a][contains(., 'Применить')]"),
    ]
    click_first(driver, apply_locators, timeout=5)
    time.sleep(2)
    make_screenshot(driver, "3. step3_promo_applied.png")

    if text_exists(driver, "не найден") or text_exists(driver, "неверн") or
text_exists(driver, "ошиб"):
        print("[ SUCCESS ] Scenario 3 done")
    else:
        raise AssertionError("Сообщение об ошибке для промокода не
появилось")

def main():
    print("Запуск тестов для Dodo Pizza...")
    driver = setup_driver()
    if not driver:
        return

    try:
        run_scenario_1(driver)
        run_scenario_2(driver)
        run_scenario_3(driver)
        print("\n[ SUCCESS ] All scenarios completed!")

```



```

except Exception as e:
    print(f"\n[ FATAL ERROR ] {e}")
finally:
    print("[ INFO ] Done. Browser stays open.")

if __name__ == "__main__":
    main()

```

Сценарий 1 (позитивный): Добавление пиццы в корзину с выбором размера

Результат выполнения первого сценария представлен на рисунках 6.1-6.5.

```

PS C:\Users\ilya\Desktop\test> & c:/Users/ilya/Desktop/test/.venv/Scripts/python.exe c:/Users/ilya/Desktop/test/dodo.py
Запуск тестов для Dodo Pizza...
[ INFO ] Connecting to Chrome on port 9222...
[ SUCCESS ] Connected! URL: https://dodopizza.ru/moscow

=== Scenario 1: Add pizza to cart ===
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\1. step1_pizza_opened.png
[ INFO ] Size 35 cm selected
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\1. step2_size_35.png
[ INFO ] Added to cart
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\1. step3_added_to_cart.png
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\1. step4_cart_opened.png
[ SUCCESS ] Scenario 1 done

```

Рисунок 6.1 – Консоль вывода

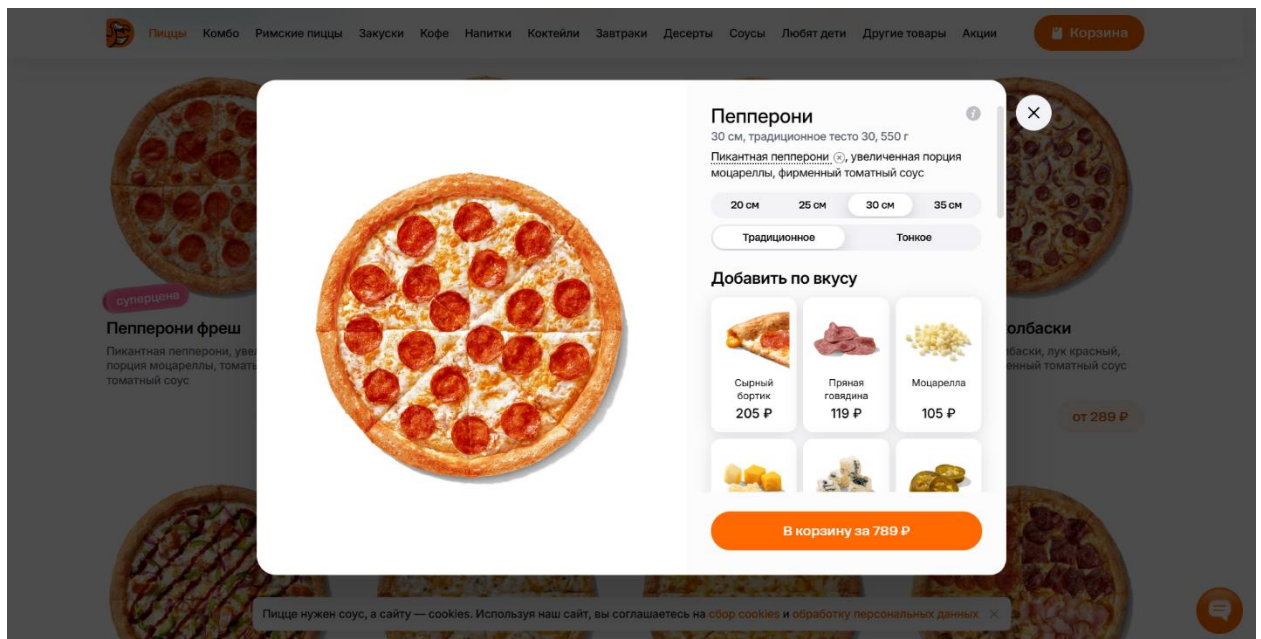


Рисунок 6.2 – Выбор пиццы Пепперони

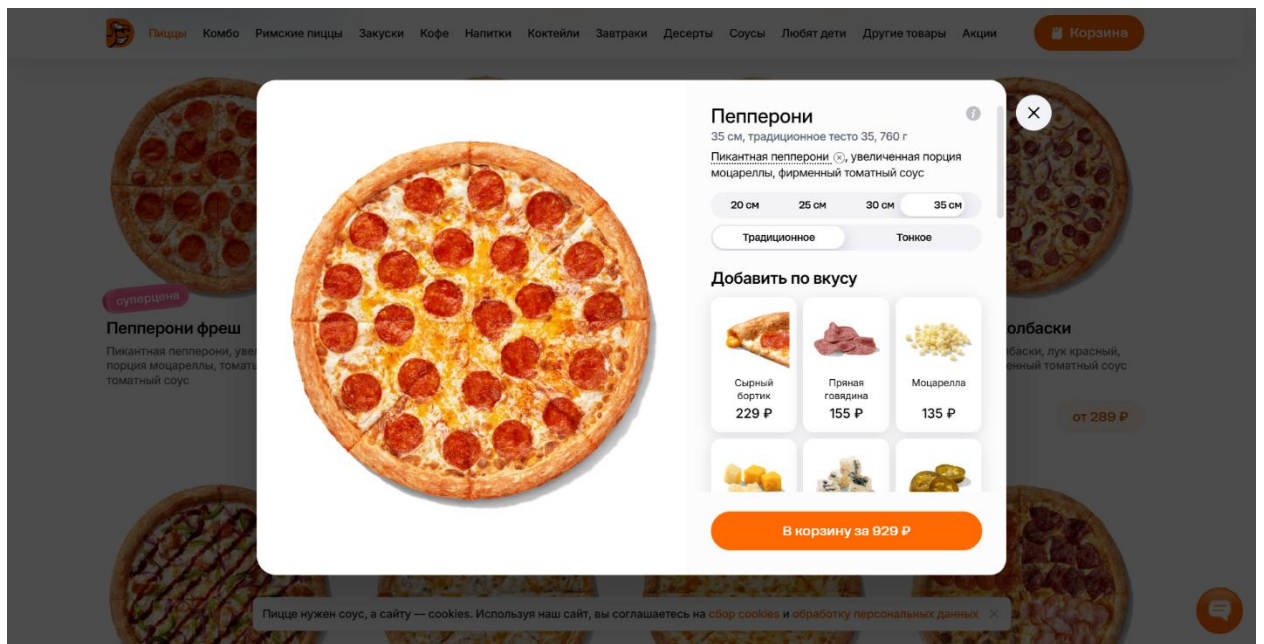


Рисунок 6.3 – Выбор размера 35 см

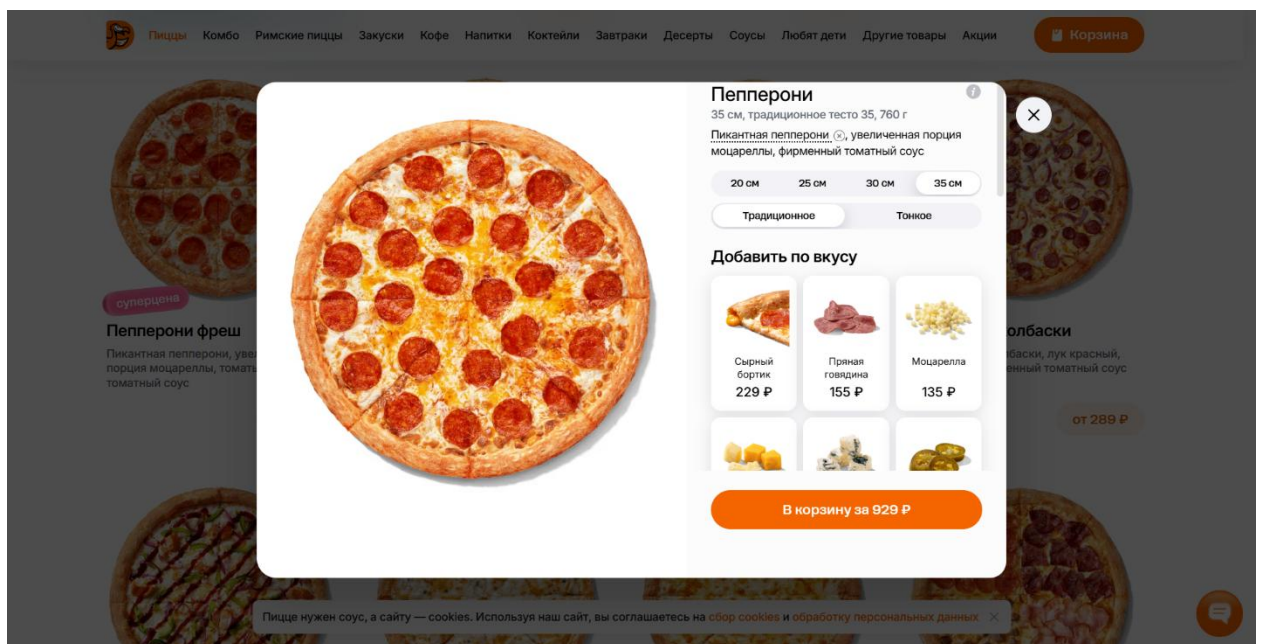


Рисунок 6.4 – Добавление в корзину

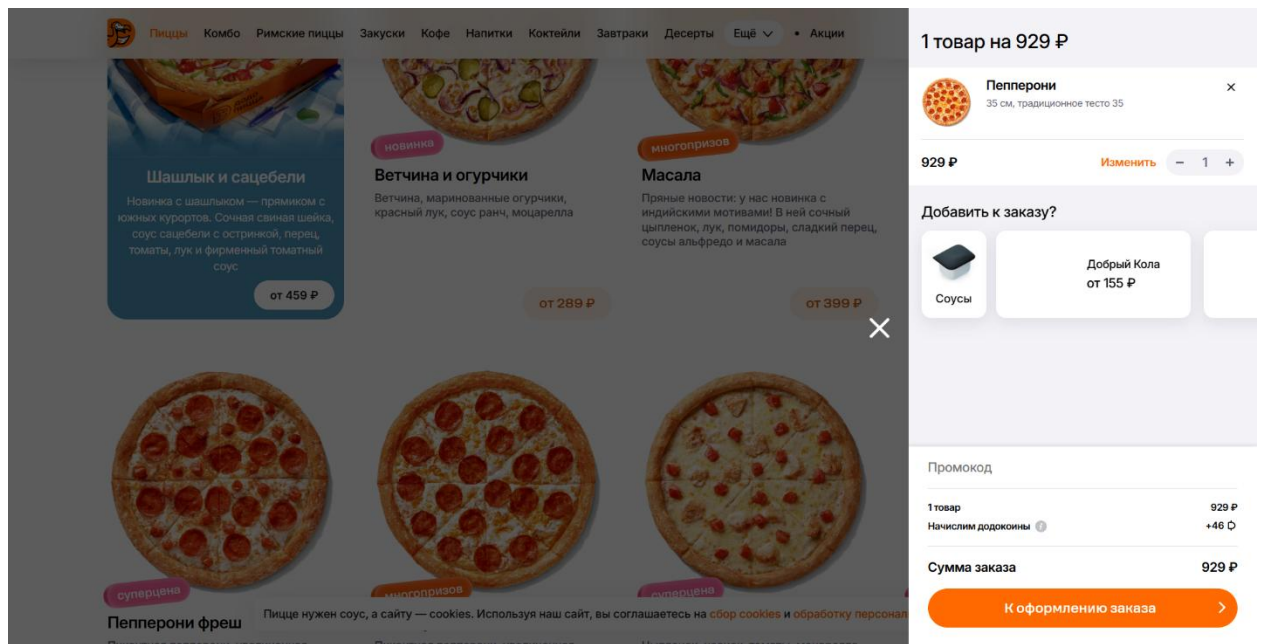


Рисунок 6.5 – Пицца Пепперони в корзине

Сценарий 2 (позитивный): Выбор города и проверка изменения доступности товаров

Результат выполнения первого сценария представлен на рисунках 6.6-6.9.

```
=== Scenario 2: Change city ===
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\2. step1_city_popup_opened.png
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\2. step2_city_typed.png
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\2. step3_city_selected.png
[ SUCCESS ] Scenario 2 done
```

Рисунок 6.6 – Консоль вывода

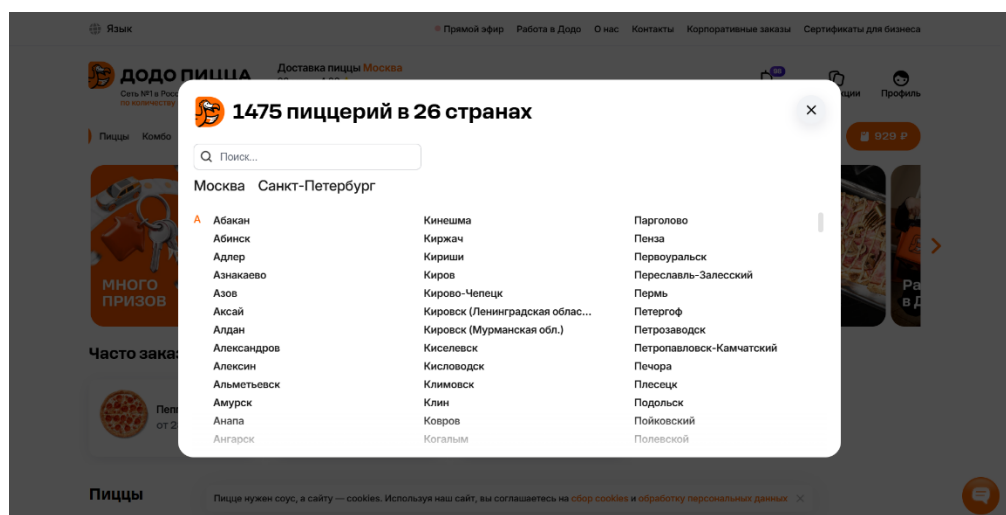


Рисунок 6.7 – Открытие списка городов для выбора

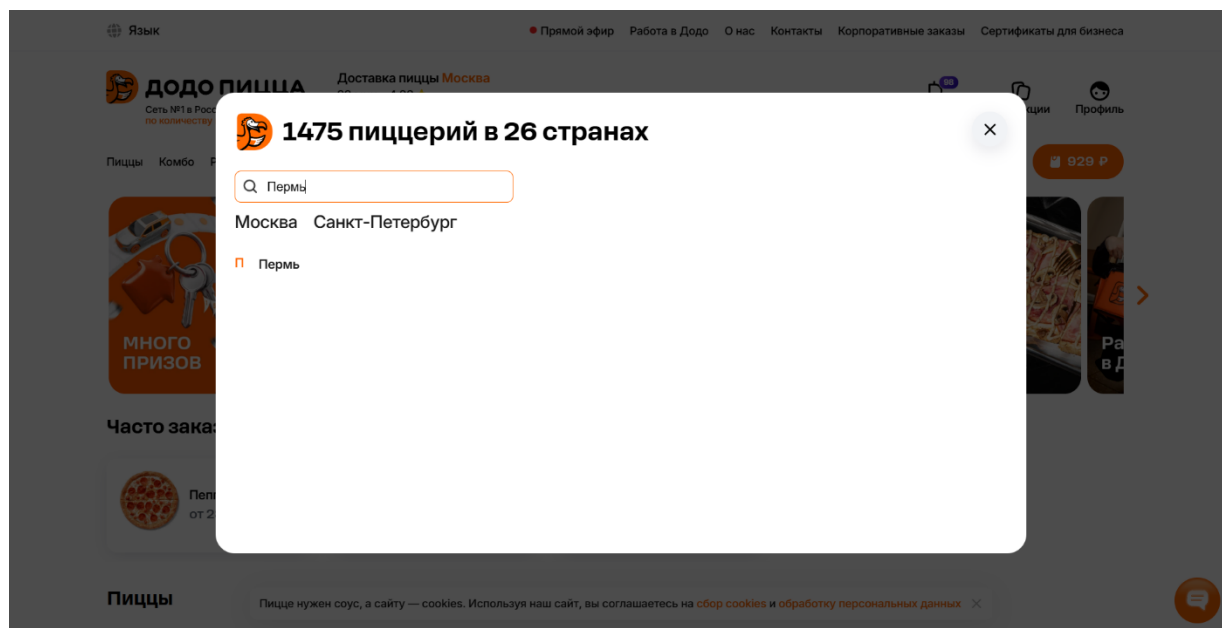


Рисунок 6.8 – Ввод города «Пермь»

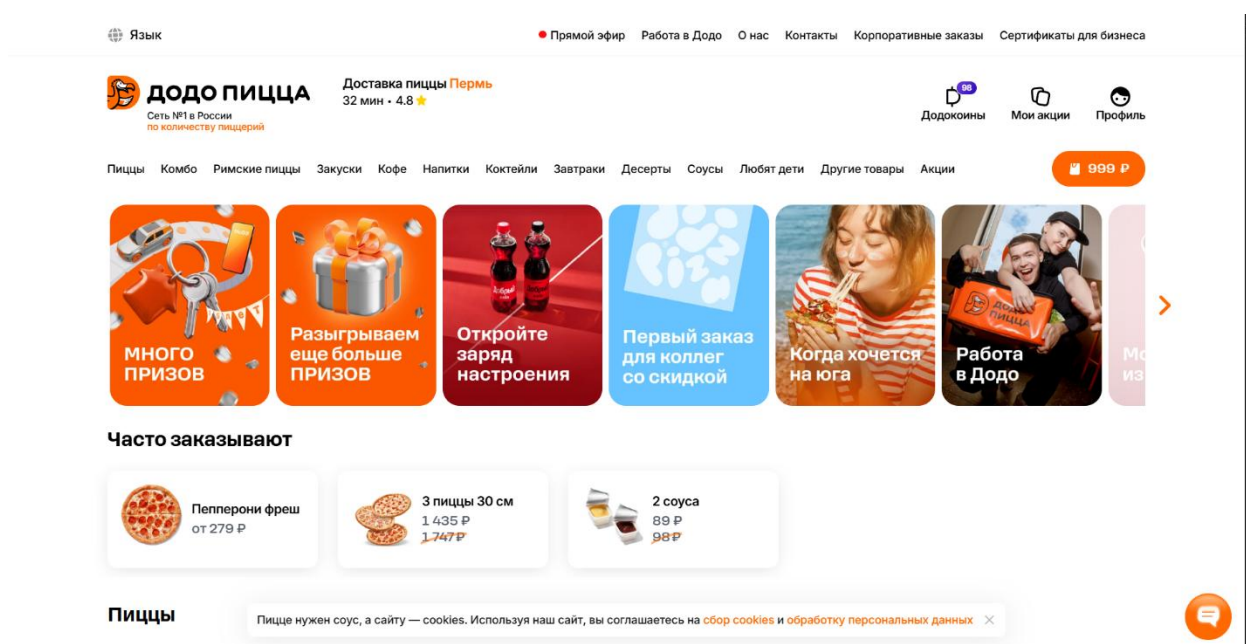


Рисунок 6.9 – Отображение нового выбранного города

Сценарий 3 (негативный): Попытка активации несуществующего промокода

Результат выполнения первого сценария представлен на рисунках 6.10-6.13.


```

=== Scenario 3: Invalid promo code ===
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\3. step1_cart_for_promo.png
[ INFO ] Promo code entered
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\3. step2_promo_typed.png
[ INFO ] Screenshot: c:\Users\ilya\Desktop\test\screenshots\3. step3_promo_applied.png
[ SUCCESS ] Scenario 3 done

[ SUCCESS ] All scenarios completed!
[ INFO ] Done. Browser stays open.
PS c:\Users\ilya\Desktop\test>

```

Рисунок 6.10 – Консоль вывода

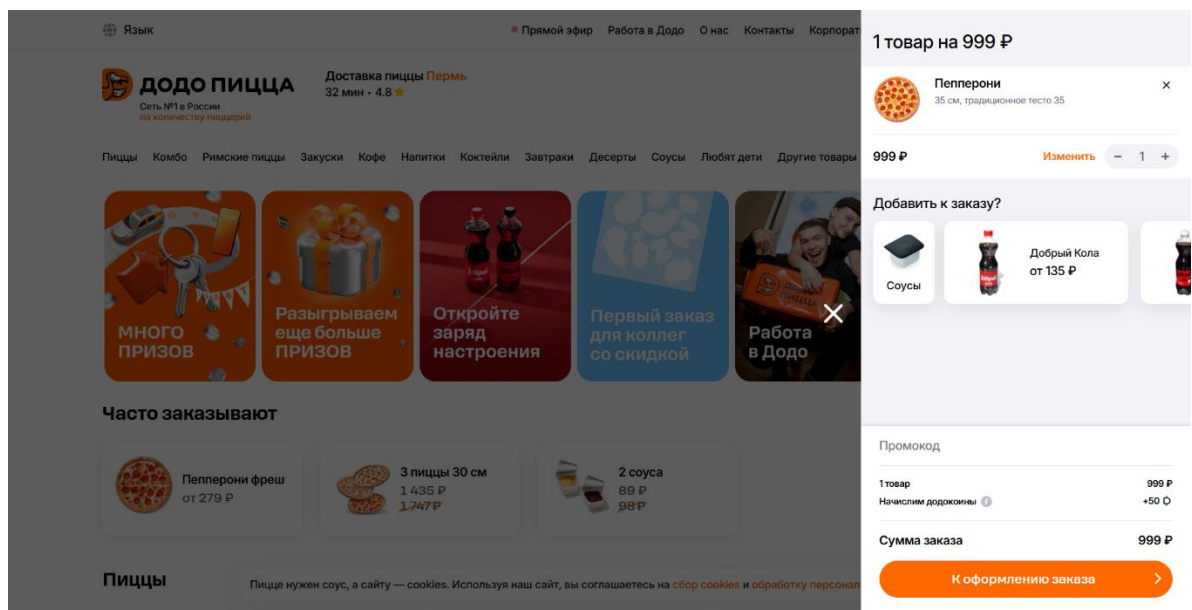
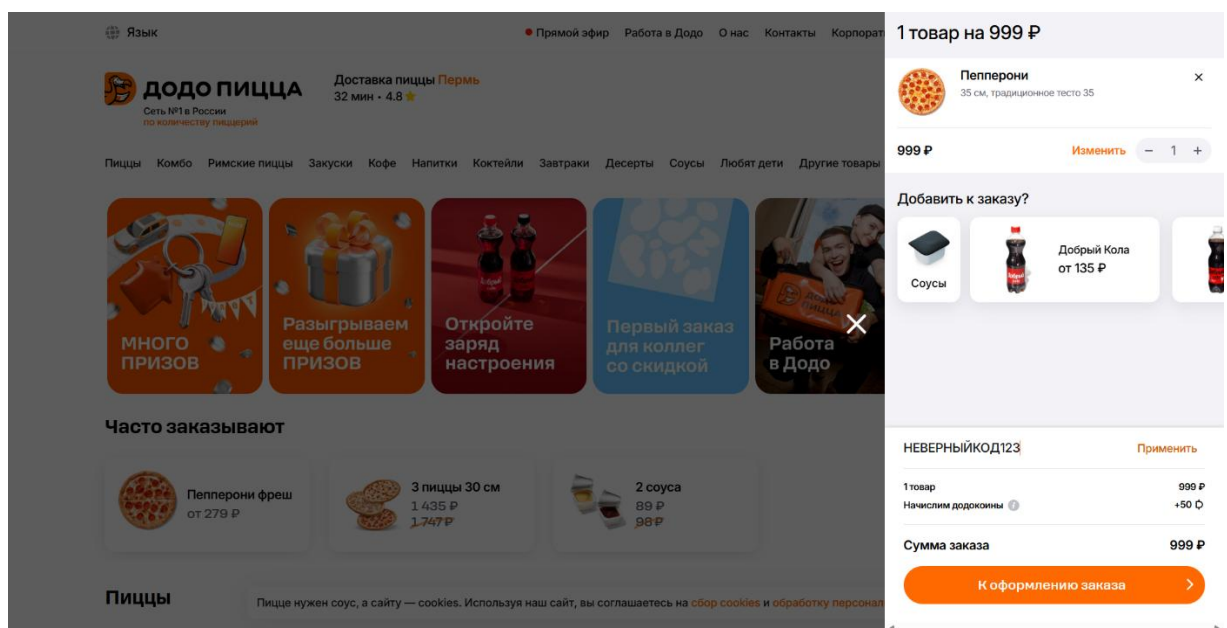


Рисунок 6.11 – Пицца Пепперони в корзине



6.12 – Автоматический ввод неверного промокода

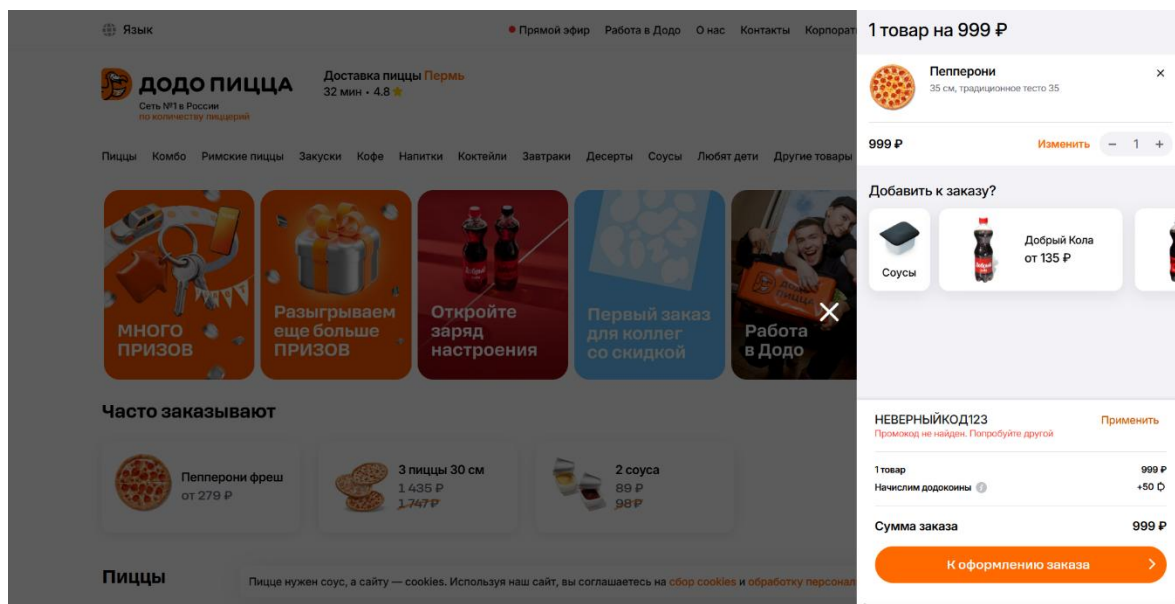


Рисунок 6.13 – Результат неверно введенного промокода

4. Общий итог тестирования

Сценарий	Тип	Статус	Примечание
Добавление пиццы в корзину	Позитивный	Пройден	Все шаги выполнены, товар добавился корректно.
Смена города	Позитивный	Пройден	Город изменился, меню загрузилось без ошибок.
Ввод неверного промокода	Негативный	Пройден	Система выдала сообщение об ошибке, сумма не изменилась.

Вывод: Основной функционал сайта Додо Пицца работает стабильно. Ошибок не обнаружено. Тестирование показало, что сайт корректно

обрабатывает как позитивные действия пользователя, так и негативный сценарий с неверным промокодом.